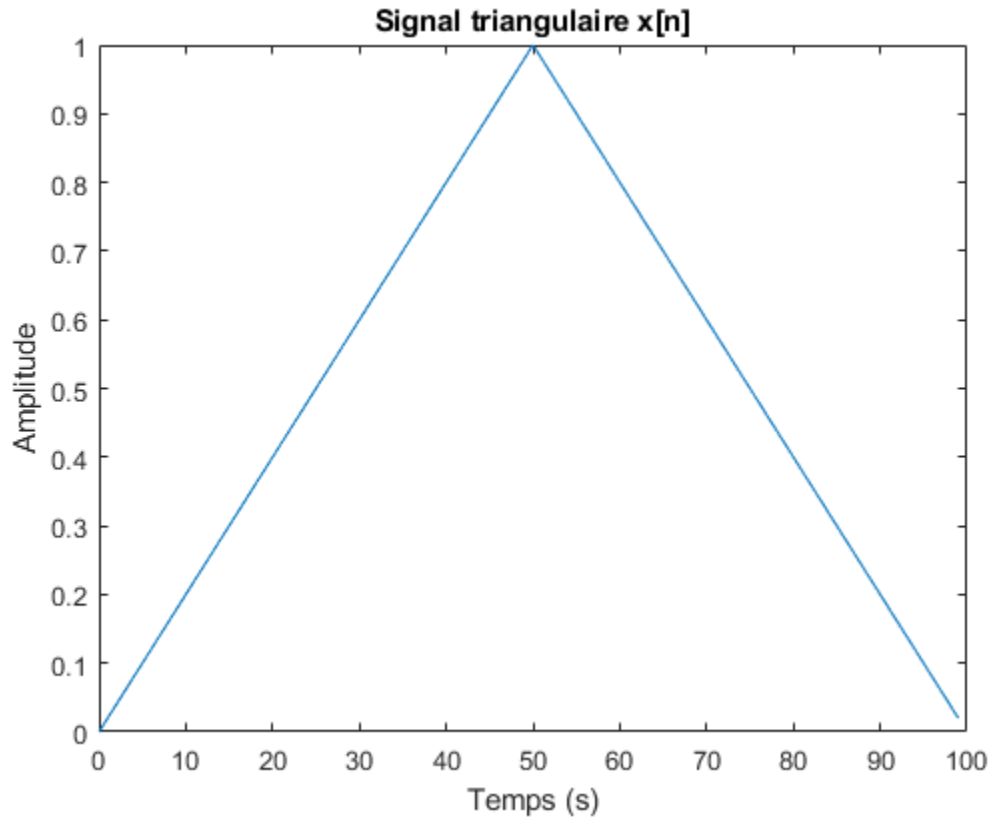

```
clear all;  
close all;
```

Construction du jeu de données

Simulation du signal d'entrée

```
L = 100;          % durée d'observation du signal d'entrée (secondes)  
fe = 1;          % fréquence d'échantillonnage (Hz)  
Te = 1/fe;       % période d'échantillonnage (secondes)  
Nx = L/Te;       % nombre d'échantillons du signal x  
nx = 0:Nx-1;     % indice temporel (entier)  
tx = nx*Te;      % base de temps (secondes)  
  
%Question 1  
%On créer le vecteur x à vide rempli de valeur 0 afin de s'asssurer de la  
%bonne taille du vecteur  
  
L_moitie = L/2;  
  
x1 = tx(tx <= L_moitie) / L_moitie;  
  
x2 = -(tx(tx > L_moitie) - L) / L_moitie;  
  
x = [x1,x2];  
  
figure (1);  
plot(tx, x);  
title('Signal triangulaire x[n]');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```



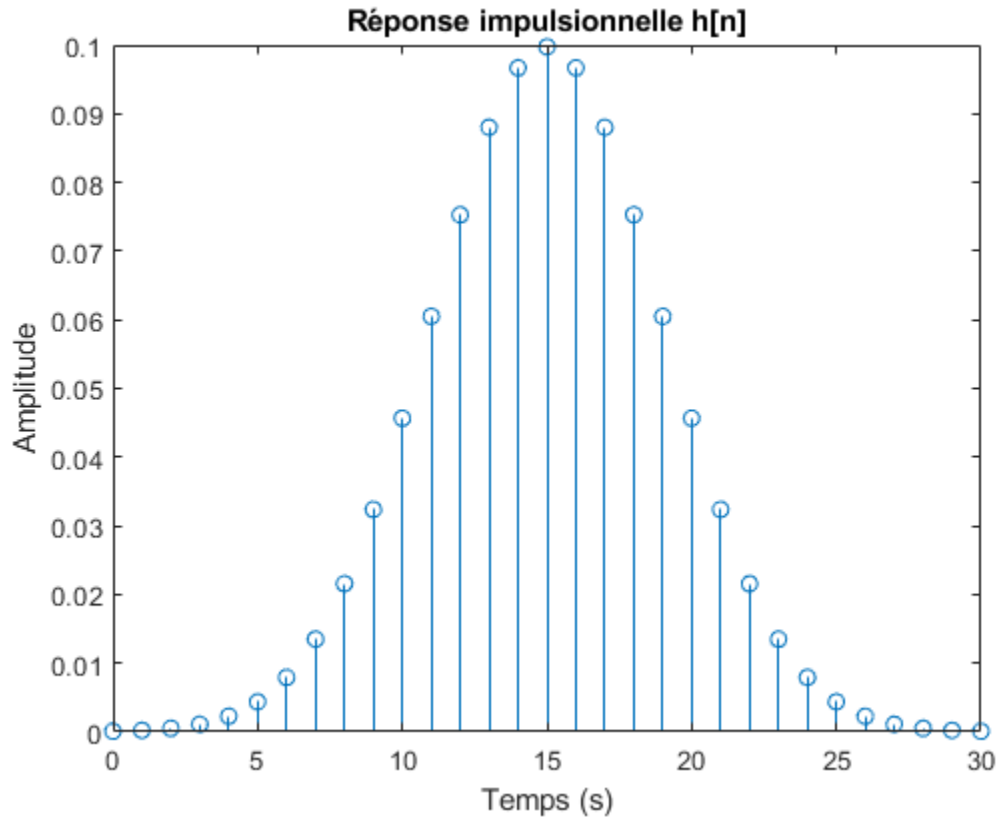
Simulation du filtre

`%Question 2`

```
m = 15;           %moyenne de la gaussienne
sigma = 4;        %écart-type de la gaussienne
Nh = 30;          % Limite den la durée de la gaussienne
Nh_samples = Nh/Te; %Nombre d'échantillons dans le filtre

t_h = 0:Nh_samples;
h = (1 / (sqrt(2 * pi) * sigma)) * exp(- (t_h - m).^2 / (2 * sigma^2));
h = h / sum(h); %normalisation pour que la somme des coefficients soit égale
à 1;

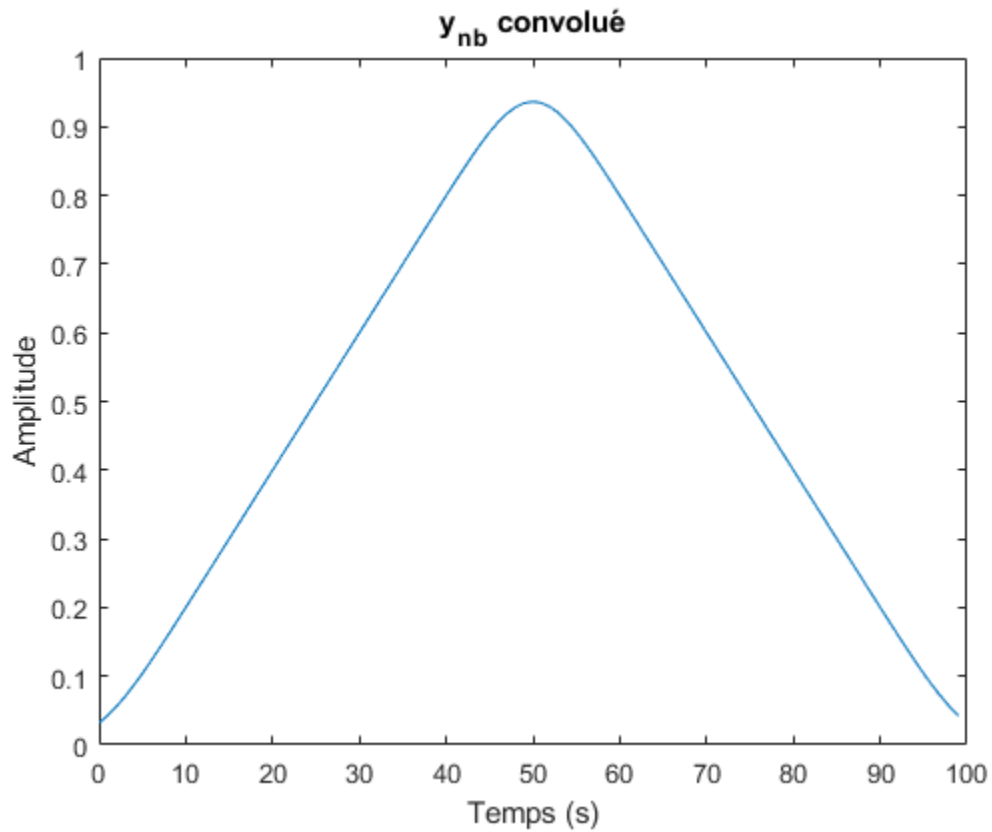
figure (2)
stem(t_h, h);
title(' Réponse impulsionnelle h[n] ');
xlabel('Temps (s)');
ylabel('Amplitude');
```



Simulation du problème direct : première approche

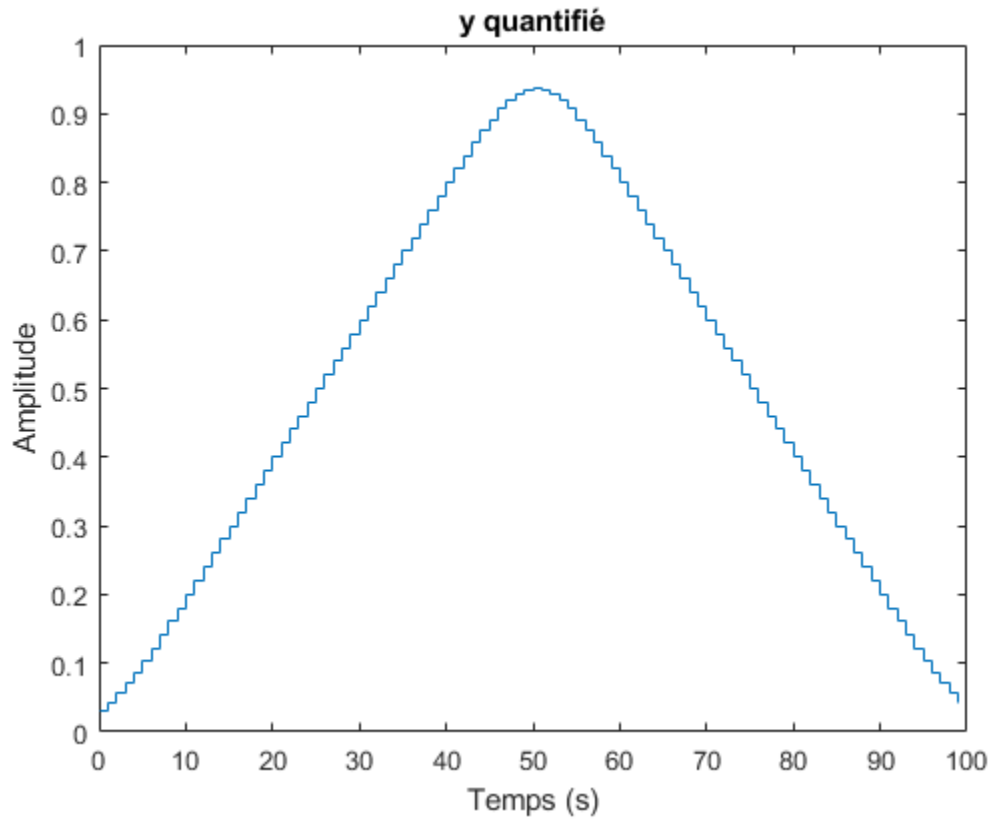
```
%Convolution du signal d'entrée x[n] avec la réponse impulsionnelle h[n]
%Question 5
y_nb = conv(x, h, 'same'); %'same' permet de garder la même longueur que x
% sans ça la taille de y_nb est 1*130 ce qui ne me permet pas de l'afficher
% puisque tx à une taille de 1*100 (et ne respecte l'observation de 100s)

figure(3);
plot(tx, y_nb);
title('y_n_b convolué');
xlabel('Temps (s)');
ylabel('Amplitude');
```



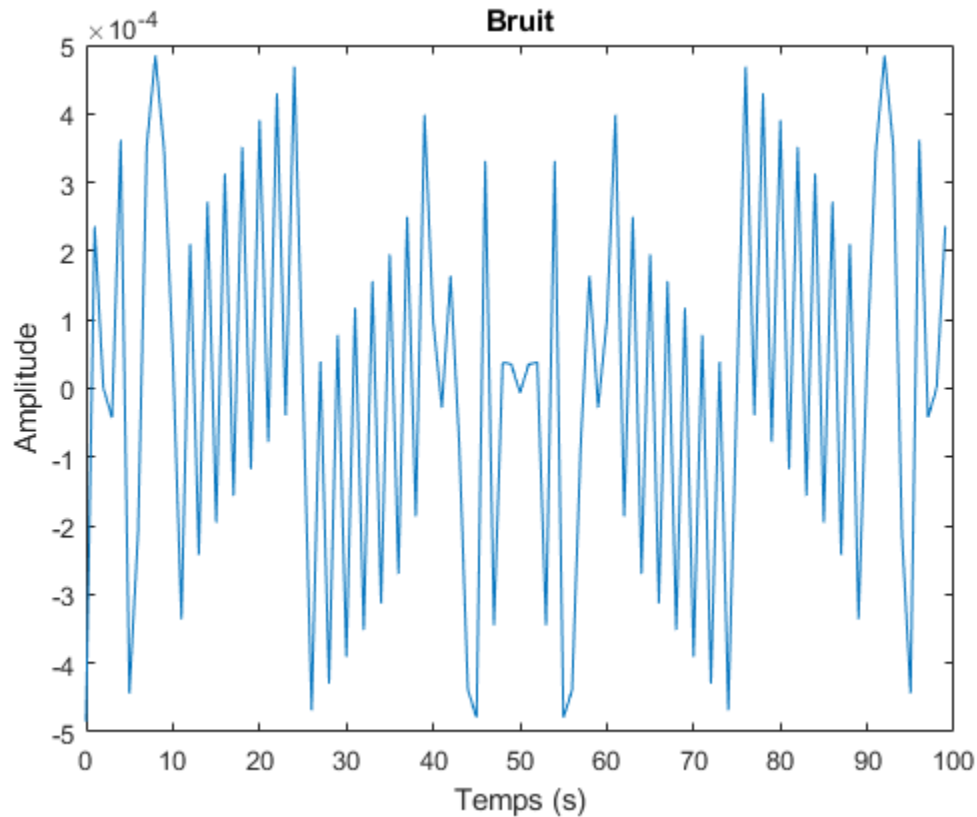
```
%Question 6
y_quantifie = round(y_nb * 2^10) / 2^10;

figure (4);
stairs(tx, y_quantifie);
title('y quantifié');
xlabel('Temps (s)');
ylabel('Amplitude');
```



```
%Question 7
b = y_quantifie - y_nb;

figure(5);
plot(tx, b);
title('Bruit');
xlabel('Temps (s)');
ylabel('Amplitude');
```



```
% Simulation du problème direct par approche matricielle
%Question 8
% ynb = Hx
Ny = Nx;
%Car le signal est tronqué et que nous avons utilisé l'option 'same' pour
%faire la convolution il est donc normal d'avoir la même taille pour la
%création de la matrice de toeplitz
disp(['La dimension du vecteur y_nb[n] est : ', num2str(Ny)]);

%Question 9

%Création du premier vecteur de la première colonne avec des valeurs nulles
% (réponse impulsionnelle)
coll = [h, zeros(1, Ny-Nh-1)];

%Création du premier vecteur de la première ligne avec des valeurs nulles
% (signal d'entrée)
lig1 = [h(1), zeros(1, Ny-1)];

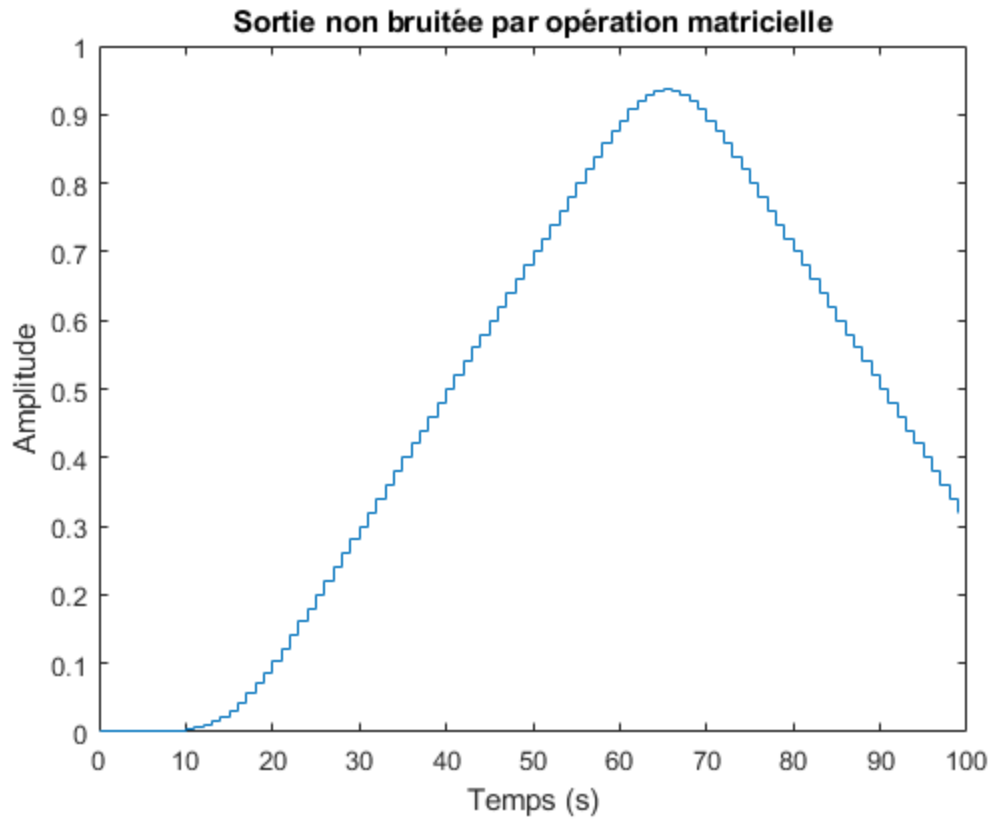
H = toeplitz(coll, lig1);

%Question 10
ynb_matrix = H*x';

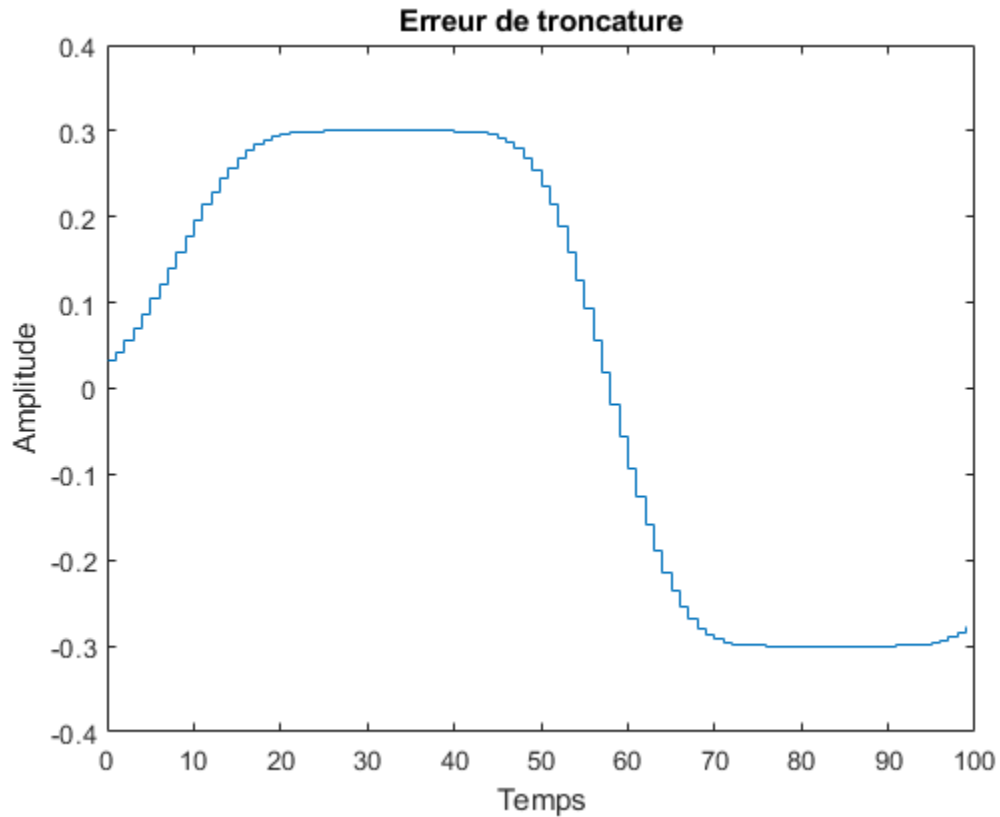
figure (6);
```

```
stairs(tx, ynb_matrix);  
title('Sortie non bruitée par opération matricielle');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```

La dimension du vecteur $y_nb[n]$ est : 100



```
e_trunc = y_nb - ynb_matrix';  
figure(7);  
stairs(tx,e_trunc);  
title('Erreur de troncature');  
xlabel('Temps');  
ylabel('Amplitude');
```

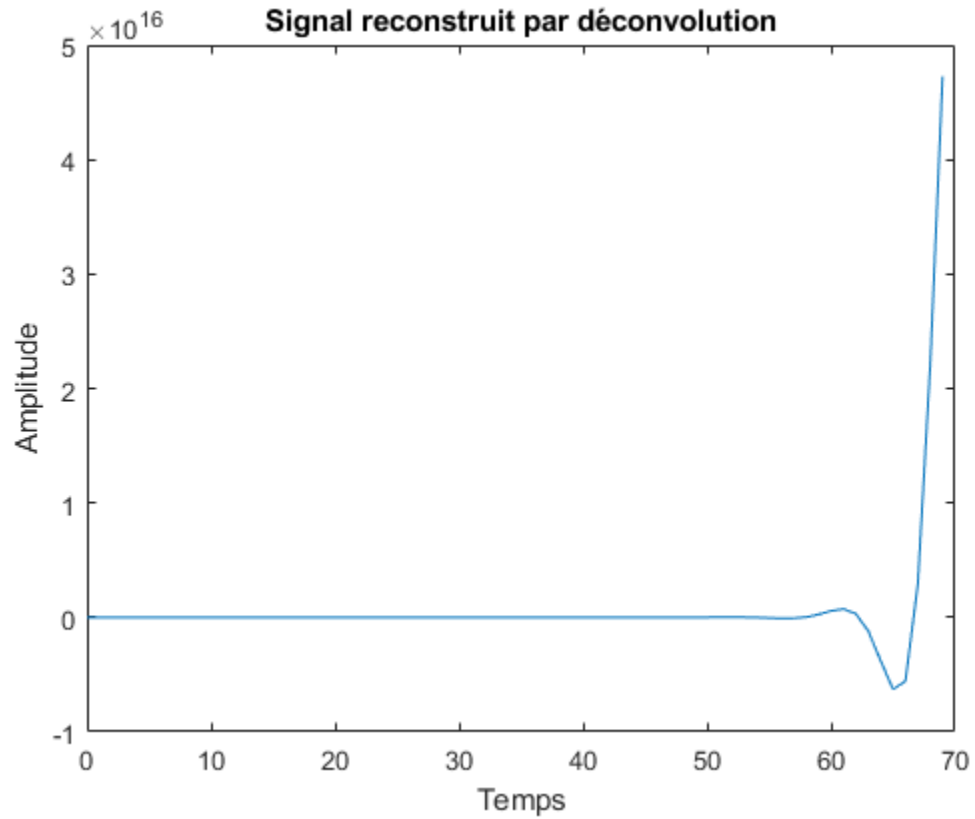


Déconvolution

Déconvolution "naïve" et constat de l'instabilité

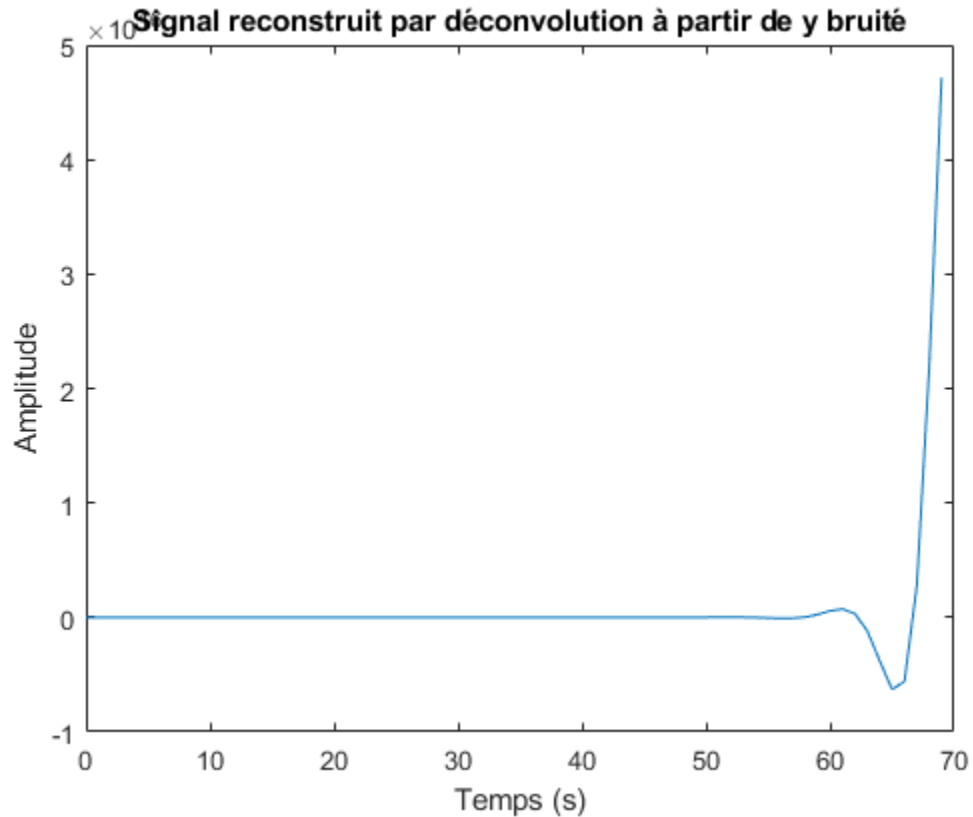
`%Question 11`

```
[x_rec_nb, reste] = deconv(y_nb, h);  
tx_deconv = tx(1:length(x_rec_nb));  
  
figure (8);  
plot(tx_deconv ,x_rec_nb);  
title('Signal reconstruit par déconvolution');  
xlabel('Temps');  
ylabel('Amplitude');
```

%Question 12

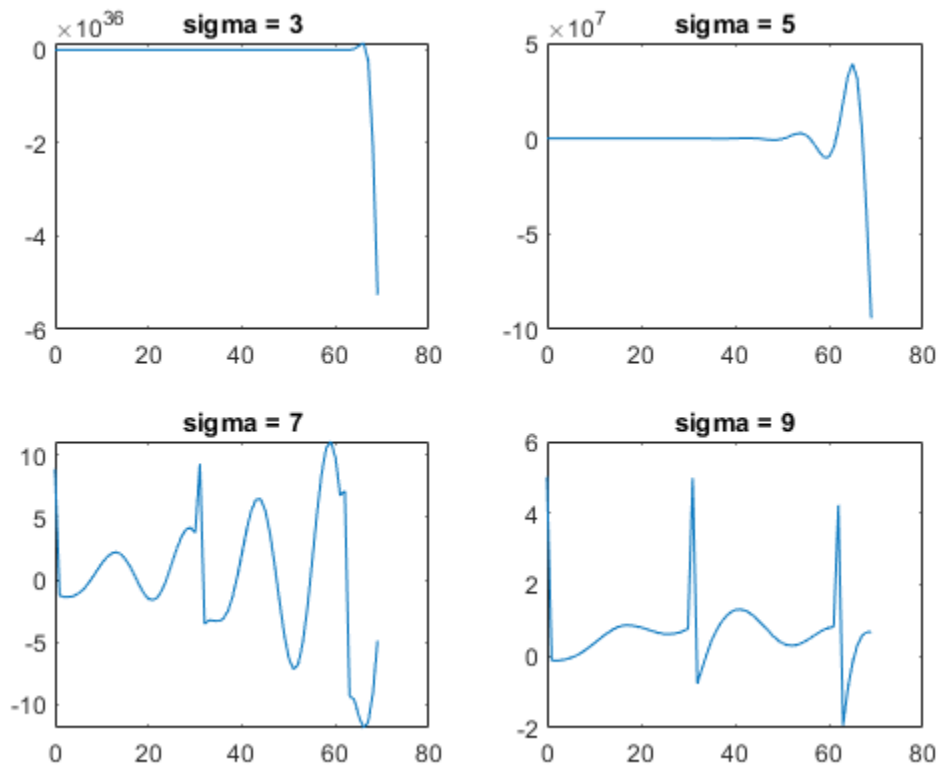
```
[xrec, reste_bruit] = deconv(y_quantifie,h);  
figure(9);  
plot(tx_deconv,xrec);  
title('Signal reconstruit par déconvolution à partir de y bruité');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```



%La déconvolution du signal bruité et du signal non bruité ne ressemble pas
 %au signal d'entrée triangulaire, cela peut être du à plusieurs facteurs
 %comme le fait que notre signal est tronquée lors de la convolution avec
 %l'option 'same'. Cela peut aussi être du à la déconvolution elle même, on
 %sait que la déconvolution amplifie les erreurs et le bruit

%Question 13

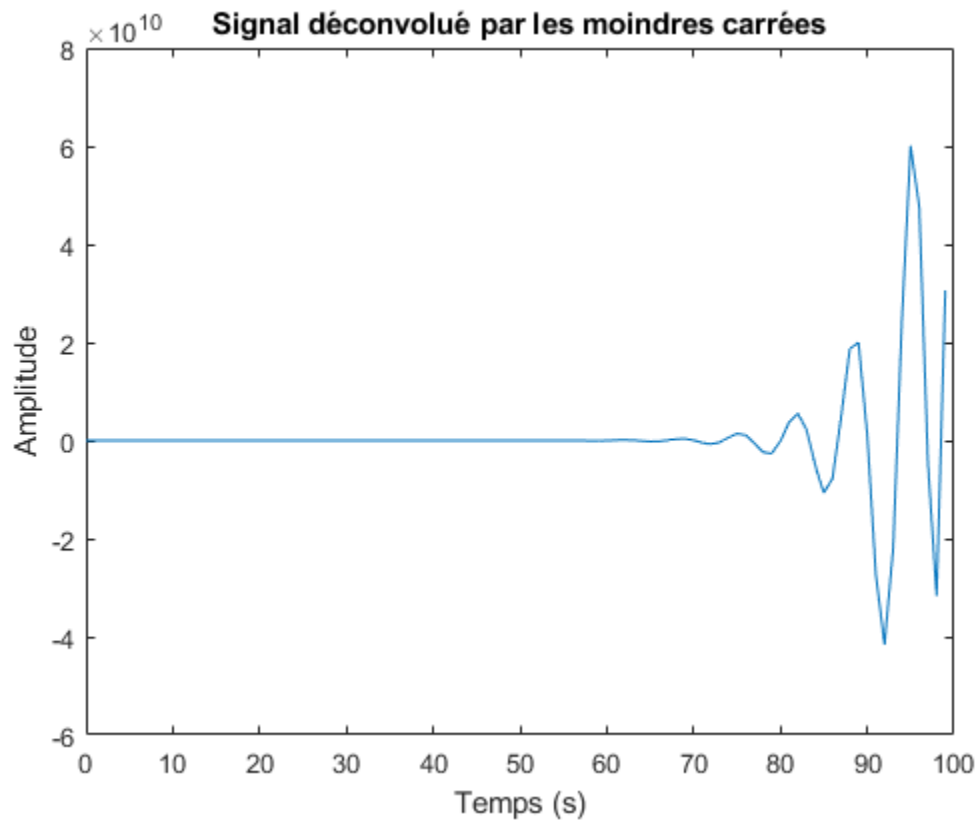
```
figure(10);
sigma_range = [3 5 7 9];
for i = 1:length(sigma_range)
    sigma=sigma_range(i);
    ht = (1 / (sqrt(2 * pi) * sigma)) * exp(- (t_h - m).^2 / (2 * sigma^2));
    ht = ht / sum(ht);
    y_nbt = conv(x, ht, 'same');
    [x_rec_nb, reste] = deconv(y_nbt, ht);
    tx_deconv = tx(1:length(x_rec_nb));
    subplot(2,2,(i));
    plot(tx_deconv,x_rec_nb);
    title(['sigma = ' num2str(sigma_range(i))]);
end
```



```
% Déconvolution par moindres carrées
%Question 14
```

```
x_rec_ls = pinv(H)* (H'*y_quantifie(:));
```

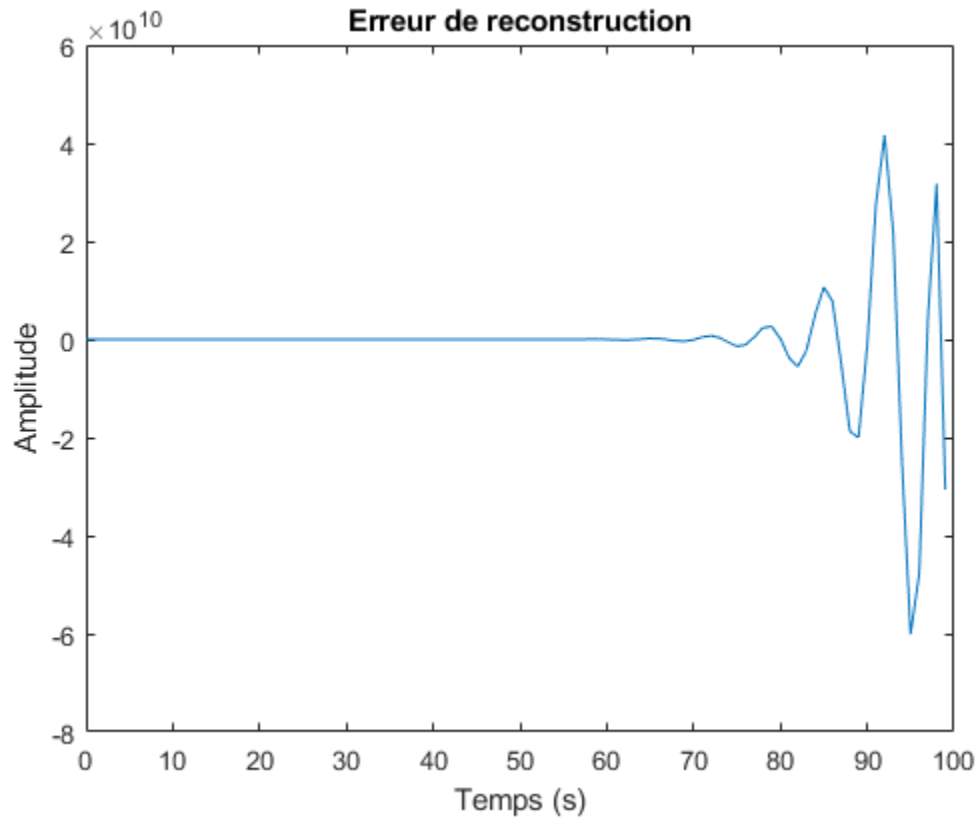
```
figure(11);
plot(tx, x_rec_ls);
title('Signal déconvolué par les moindres carrées');
xlabel('Temps (s)');
ylabel('Amplitude');
```



%Question 15

```
erreur_rec = x - x_rec_ls';  
energie_erreur_rec = sum(erreur_rec.^2);  
disp(['L énergie de reconstruction est : ', num2str(energie_erreur_rec)]);  
figure(12);  
plot(tx,erreur_rec);  
title('Erreur de reconstruction');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```

L énergie de reconstruction est : 1.24480345591469e+22



%Question 16

%Calcul des valeurs singulières de H grâce à la SVD

```
[U, S, V] = svd(H);
```

%Extraction des valeurs singulières par diagonalisation de S

```
val_sing = diag(S);
```

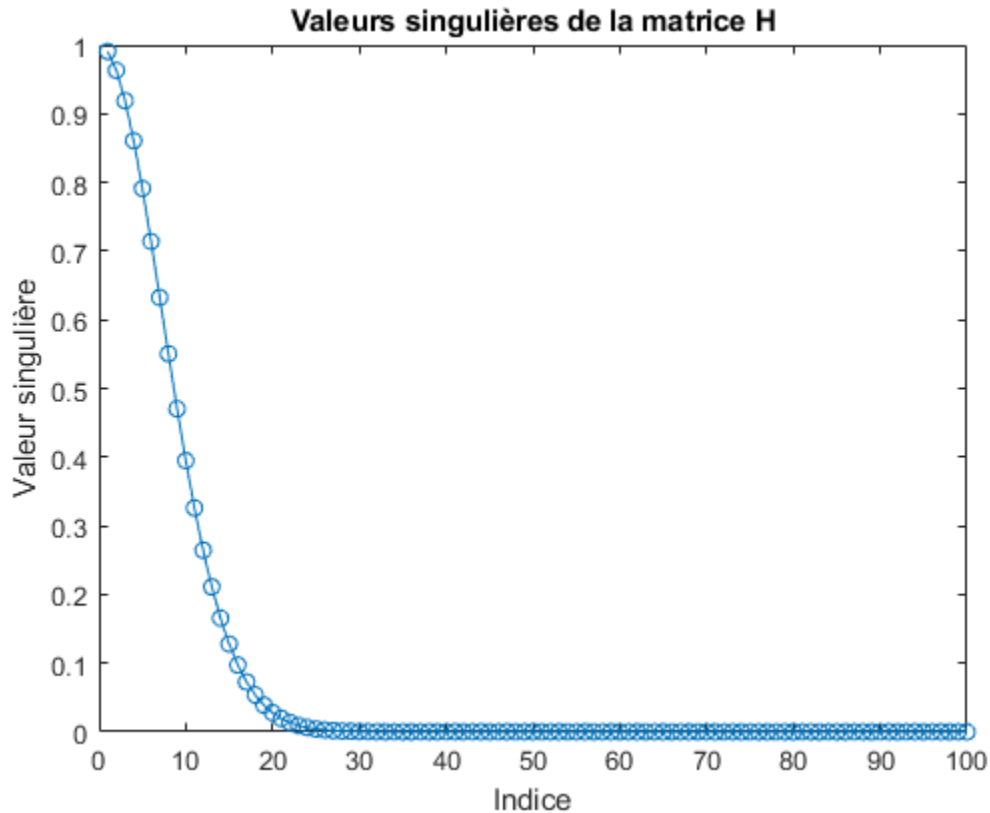
```
figure(13);
```

```
plot(val_sing, 'o-');
```

```
title('Valeurs singulières de la matrice H');
```

```
xlabel('Indice');
```

```
ylabel('Valeur singulière');
```



```
%Calcul du conditionnement de H
%Le conditionnement est le ratio entre la plus grande et la plus petite
%valeur singulière
max_val_sing = max(val_sing);
min_val_sing = min(val_sing);

cond_H = max_val_sing / min_val_sing;
disp(['Le conditionnement de H est : ', num2str(cond_H)]);

%Question 17

% Le conditionnement de la matrice de convolution H est influencé par la
% forme de la réponse impulsionnelle du filtre, qui dépend du paramètre
% sigma.

% Lorsque sigma devient grand, la réponse impulsionnelle devient plus lisse
% et plus étendue. Cela entraîne une diminution des valeurs singulières de
% H et un conditionnement plus élevé (mal conditionné). Ce qui rend la
% solution des moindres carrés moins stable et plus sensible au bruit.

% Lorsque sigma est petit, la réponse impulsionnelle h devient plus
% concentrée autour du paramètre m. Cela entraîne une augmentation des
% valeurs singulières et un conditionnement plus faible. Ce qui rend la
% solution plus stable, la rendant moins sensible au bruit.

% Déconvolution par régularisation linéaire quadratique
```

%Question 18

```
dcoll = zeros(1, Nx);
dcoll (1:2) = dcoll (1:2) + [1 -1];
dlig1 = zeros(1, Nx);
dlig1 (1 ,1) = dcoll(1,1);
D1 = toeplitz(dcoll,dlig1);

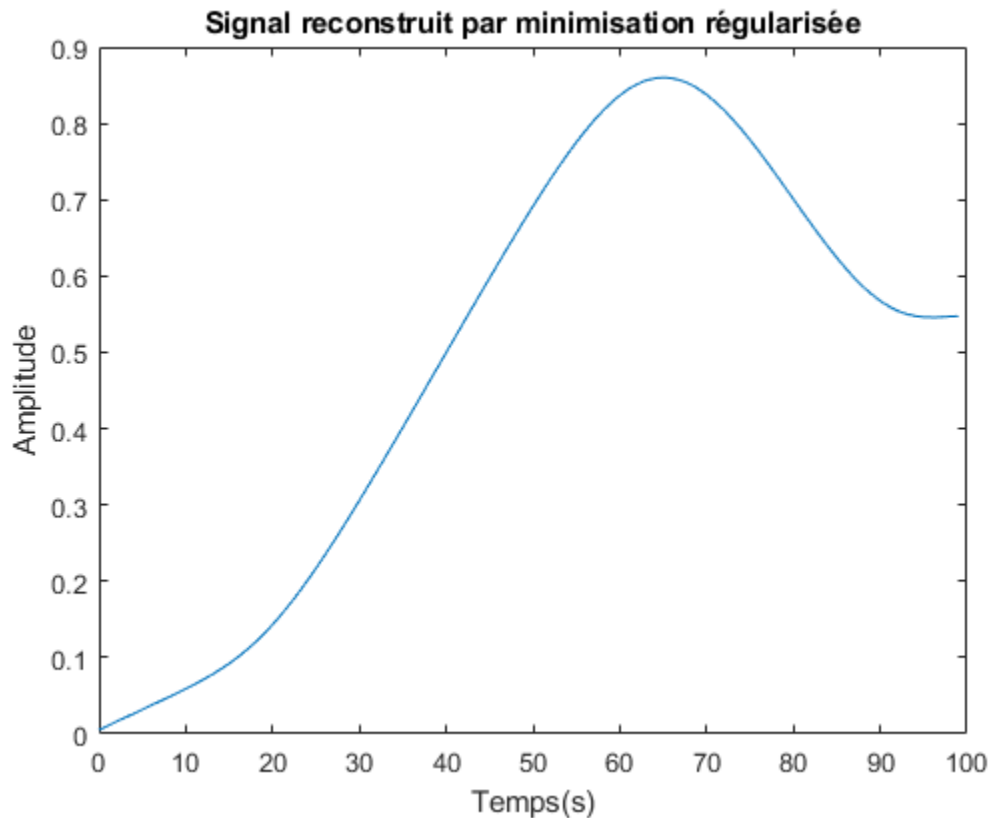
lambda = 40.6781; % valeur optimal récupéré plus tard dans le TP

HTH = H * H';
lambdaD1tD1 = lambda * (D1' * D1);
HY = H * y_nb';

%Identification de x
x_reg = (HTH + lambdaD1tD1) \ HY ;

figure(14);
plot(tx, x_reg);
title('Signal reconstruit par minimisation régularisée');
xlabel('Temps(s)');
ylabel('Amplitude');
```

Le conditionnement de H est : 1.757189367402437e+16

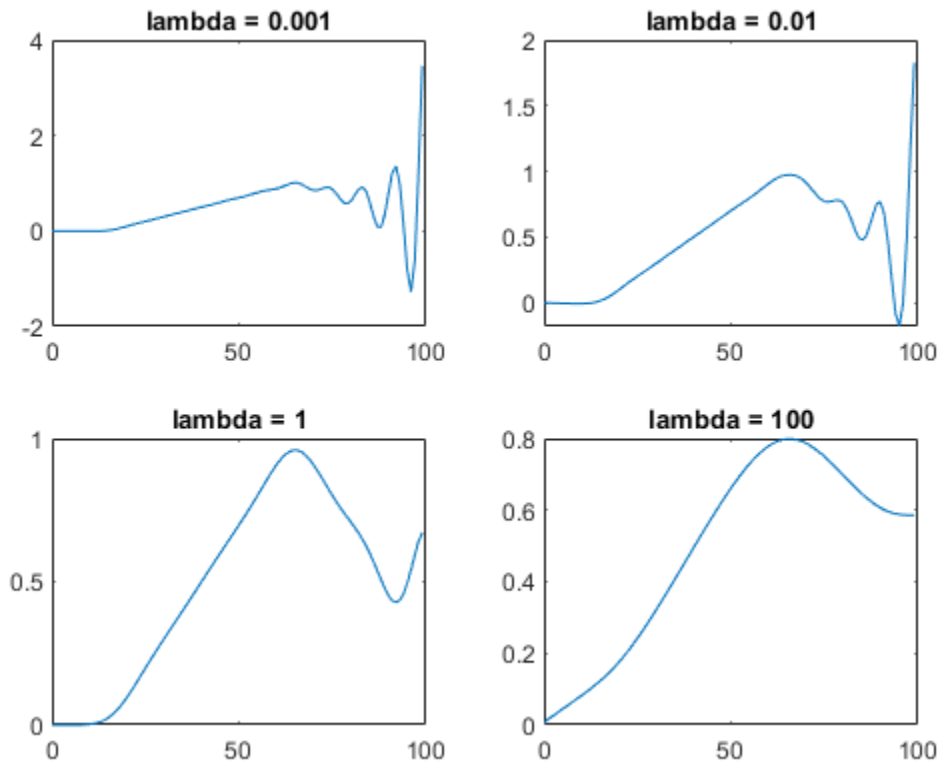


%Question 19
%Recherche de manière empirique

```

lambda_range = [0.001 0.01 1 100];
figure (15);
for i = 1:length(lambda_range)
    x_reg = (HTH + lambda_range(i) * (D1' * D1)) \ HY;
    subplot(2,2,i);
    plot(tx,x_reg);
    title(['lambda = ' num2str(lambda_range(i))]);
end

```



```

%Question 20
lambda_val = logspace(-7, 2, 10000);
errors = zeros(1, length(lambda_val));
for i = 1:length(lambda_val)
    lambda = lambda_val(i);
    x_reg = (HTH + lambda * (D1' * D1)) \ HY;
    errors(i) = sum((x_reg - x').^2);
end
[~, idx_min] = min(errors);
lambda_opt = lambda_val(idx_min);
disp(['Le meilleur lambda est : ', num2str(lambda_opt)]);

```

%J'ai eu beaucoup de difficulté à faire ce TP, tous d'abord car je ne
 %considère pas avoir les pré-requis en terme de connaissance et de
 %compétence pour cette UE. Afin de pallier à cela, j'ai utilisé les cours de

%M. Djhermoune, le site de matlab et chatgpt afin de m'aiguiller sur
%certaines fonctions à utiliser ou manière d'écrire certains morceaux
%d'algorithme. Enfin, j'ai discuté avec M. Dausse et M. Erebi de mes
%résultats afin de voir si j'étais totalement dans l'erreur.

Le meilleur lambda est : 40.6781

Published with MATLAB® R2024a